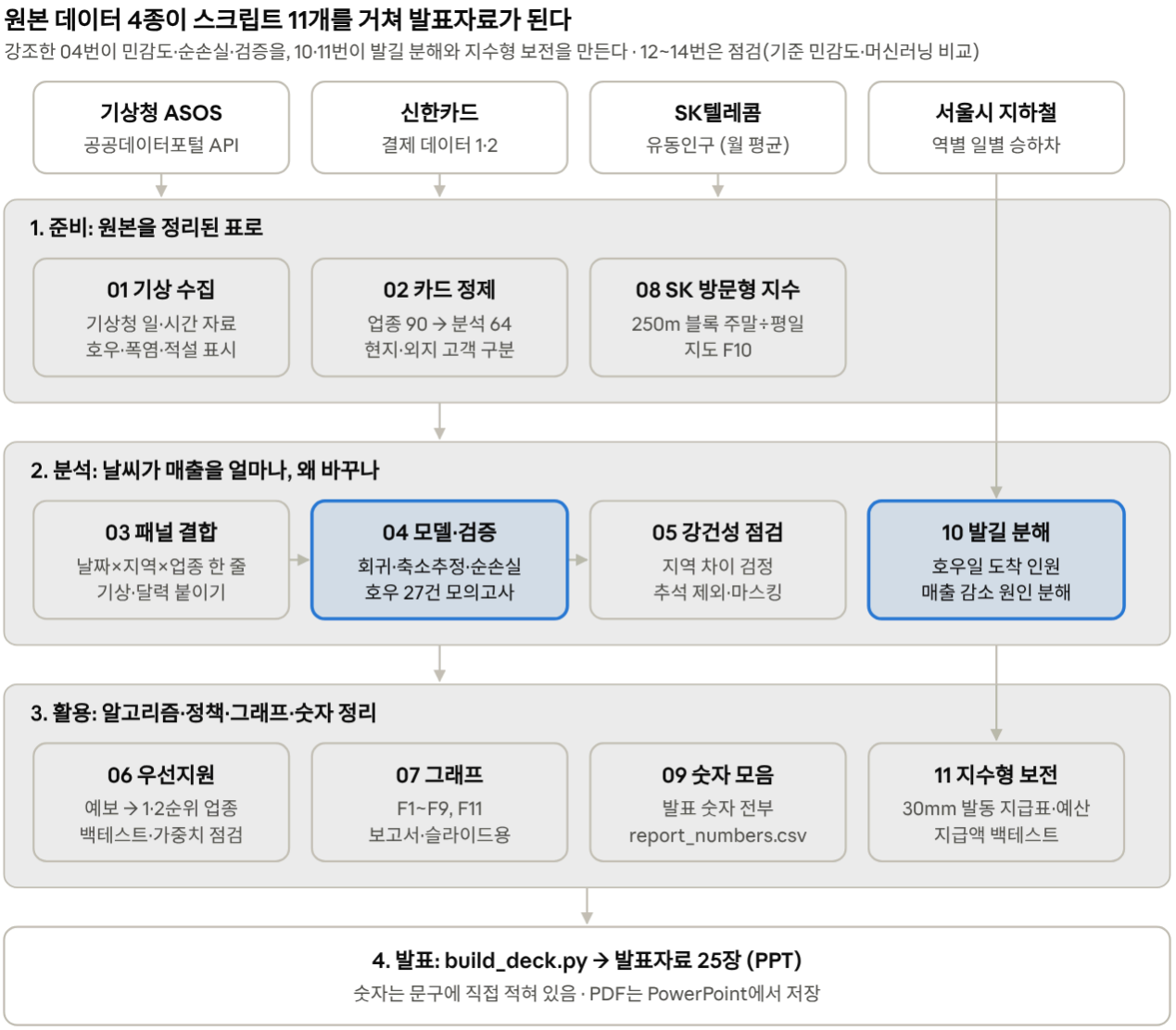
같은 비, 다른 피해 — 수정을 위한 상세 해설서

Oct 24, 2026 · @전건우

이 문서는 분석의 모든 숫자가 어디서 어떻게 나오는지를 실제 값으로 따라가며, 어느 파일의 무엇을 고치면 결과가 어떻게 달라지는지 설명한다. 최종 발표자료(25장)는 '춘천은 발길이 끊기고, 강남은 지갑이 닫힌다'를 뼈대로 하고 호우 지수형 손실보전을 정책으로 붙였다. 새로 더해진 두 단계는 9-1장(지하철)과 9-2장(지수형 보전)에 있다. 큰 그림은 '쉽게 보는 같은 비, 다른 피해'에, 세부 계산과 수정 방법은 이 문서에 있다.

1. 전체 지도



분석 흐름 · 원본 4종, 분석 스크립트 11개 + 점검 3개(12~14), 4단계

모든 코드는 `src/` 폴더에 번호 순서대로 있고, 앞 단계의 결과 파일을 읽어 다음 결과 파일을 만든다. 중간 결과는 `data/processed/`, 분석 결과는 `outputs/`, 그래프는 `outputs/figures/`에 쌓인다. 어느 단계를 고치면 그 아래 단계를 모두 다시 실행해야 한다(11장).

2. 데이터 한 줄씩 읽기

모든 계산은 아래 네 가지 '한 줄'에서 시작한다. 실제 데이터의 한 줄을 칸별로 풀어 쓴다.

신한카드 데이터 1 (시간대별)

원본 한 줄: `20250703 12_17 서울 강남구 가구 여성 20 대 78970 5`

칸	값	뜻
TA_YMD	20250703	2025년 7월 3일
TIME_GB	12_17	낮 12시~17시 59분 (6시간 단위 4구간)
MCT_SGG_CD	서울 강남구	가게 위치
MCT_RY_CD	가구	가게 업종 (전체 90종)
SEX_CCD / AGE_CCD	여성 / 20 대	손님 성별·나이. '법인'이면 회사 카드
TS_AT	78970	결제 금액 합계(원)
USE_CNT	5	결제 건수

풀어 읽으면 "7월 3일 낮 시간에 강남 가구점들에서 20대 여성 손님이 5번, 합계 78,970원을 결제했다"는 뜻이다. 한 가게가 아니라 강남구 가구점 전체를 합한 값이다.

신한카드 데이터 2 (고객 거주지별)

원본 한 줄: `20250714 강원 춘천시 가구 강원 남성 50 대 77544 18`

시간대 대신 **CLN_SGG_CD(손님이 사는 시·도)** 칸이 있다. 가게가 춘천인데 손님이 '강원'이면 현지 고객, '서울'·'경기' 등이면 외지 고객으로 분류했다. 법인 카드는 처음부터 빠져 있다. 분석의 주 데이터는 이쪽이다.

두 파일 공통으로 알아둘 점이 있다.

- **글자 인코딩이 cp949**다. 다른 방식으로 열면 한글이 깨진다.
- **결제 5건 미만인 줄은 삭제돼 있다**(마스킹). 위 예시의 USE_CNT가 모두 5 이상인 이유다. 그래서 하루 전체가 비어 있는 업종-날짜는 '매출 0'이 아니라 '모름(NaN)'으로 두었다.

SK 유동인구

원본 한 줄(요일별 파일): `202507 | 32010350100010000001 | 1000583.99 | 1969379.15 | 1.6 | 1.55 | 2.03 | 1.98 | 2.39 | 0 | 0`

- **202507**: 2025년 7월 (월 단위라 날짜별 값이 없다)
- **BLOCK_CD**: 앞 5자리가 지역. 32010 = 춘천, 11230 = 강남. 그 외 코드는 경계에 걸친 이웃 지역이라 빼다
- **X, Y**: 50m 정사각형 칸의 위치(미터 단위 좌표, UTM-K)
- **뒷자리 7개**: 월·화·수·목·금·토·일요일의 하루 평균 유동인구. 이 칸은 평일에 사람이 있고 주말에는 없는 직장형 장소다

12월 파일은 모든 줄이 두 번 들어 있어 중복을 지운다. 시간대별·연령별 파일도 구조는 같고 뒷자리 칸만 다르다.

기상청 ASOS (가공 후)

춘천 2025년 7월 20일의 한 줄이다. 기간 중 춘천에 비가 가장 많이 온 날이다.

칸	값	뜻
rain	94.7	하루 강수량(mm)
rain_1h_max	46.6	가장 세차게 온 1시간의 강수량
tmax / tmin	32.1 / 23.0	최고·최저기온(℃)
heavy_rain	1	호우일(30mm 이상)이면 1, 아니면 0
heat33	0	폭염일(33℃ 이상)이 아님
rain_lag1	23.0	전날 강수량. lag1 = 하루 전, lag2 = 이틀 전
heavy_rain_lag3	1	사흘 전(7/17)도 호우일이었다

서울시 지하철 승하차 (외부 공공데이터)

서울 열린데이터광장 'OA-12914 지하철호선별 역별 승하차 인원 정보'의 월별 파일 6개 (`data/external/seoul_subway/CARD_SUBWAY_MONTH_2025MM.csv`)다. 한 줄은 '하루 × 노선 × 역'이고, 칸은 사용일자·노선명·역명·승차총승객수·하차총승객수·등록일자 여섯 개다. 교통카드를 찍은 사람만 센다.

- 줄 끝에 빈 칸이 하나 더 붙어 있어, pandas로 읽으면 칸이 밀린다. 코드는 csv 모듈로 읽고 앞 5칸만 쓴다.

- 춘천은 노선명이 '경춘선'이고 역명이 춘천·남춘천·김유정·강촌·백양리·굴봉산인 줄(춘천시 안의 6개 역), 강남은 경춘선이 아닌 노선에서 강남구 역 이름 목록에 든 줄이다. 목록 26개 중 1개는 자료에 없어 실제로는 25개 역이 잡힌다.
- 내린 사람(하차) 수를 지역·날짜별로 더해 '도착 인원'으로 쓴다. 하루 평균 춘천 약 6천 명, 강남 약 54만 명이다.

분석 패널 (위 세 가지를 합친 것)

`data/processed/panel_daily.csv`의 한 줄은 **'날짜 × 지역 × 업종' 하나**다. 예를 들어 '2025-07-20, 춘천, 한식' 줄에는 그날 춘천 한식점 매출(sales), 현지·외지 고객 매출(sales_local, sales_visitor), 그날 날씨(rain, heavy_rain 등), 달력 정보(요일, 공휴일 여부 등)가 한꺼번에 들어 있다. 모델은 이 줄들을 업종별로 모아 계산한다.

이 밖에 줄마다 붙은 두 가지 표시가 중요하다.

- **observed**: 그날 데이터가 있으면 1, 마스킹으로 통째 빠졌으면 0
- **reliable**: 업종 단위의 표시. 184일 중 90% 이상 관측되고, 하루 결제 건수 중간값이 30건 이상인 업종만 1. 분석은 reliable=1인 업종만 쓴다

3. 정제 규칙과 설정값

분석의 '기준'은 대부분 `src/config.py` 한 파일에 모여 있다. 숫자 하나만 바꿔도 결과가 바뀌므로, 아래 표에서 무엇을 바꾸면 어디까지 다시 실행해야 하는지 확인한다.

config.py

설정	현재 값	뜻	바꾸면 다시 실행
`HEAVY_RAIN_MM`	30	이 값 이상이면 '호우일'. 분석의 핵심 기준	01 → 03 → 04~08 → PPT
`HEAT_TMAX`	33.0	최고기온 이 값 이상이면 '폭염일' (폭염주의보 기준)	01 → 03 → 04~08 → PPT
`SNOW_MIN_CM`	0.2	새로 쌓인 눈이 이 값 이상이면 '강설일'. 0.1은 센서 흔들림이라 제외	01 → 03 → 04~08
`COLD_TMIN`	-12.0	최저기온 이 값 이하면 '한파일' (현재 모델에는 미사용)	01 → 03
`RAIN_BINS`	0.1 / 10 / 30 / 80mm	강수량 구간(없음·약·보통·강·매우강)	01 → 03

설정	현재 값	뜻	바꾸면 다시 실행
`HOLIDAYS`	8/15, 10/3, 10/5~9, 12/25	공휴일. 빠지면 공휴일 매출 변화가 날씨 효과로 섞인다	03 → 04~08
`CHUSEOK_BLOCK`	10/3~10/12	추석 연휴 기간 표시	03 → 04~08
`MIN_COVERAGE`	0.90	184일 중 이 비율 이상 데이터가 있어야 분석 대상	02 → 03 → 04~08
`MIN_MEDIAN_CNT`	30	하루 결제 건수 중간값이 이 이상이어야 분석 대상	02 → 03 → 04~08
`INDUSTRY_GROUPS`	90개 업종 → 12개 업종군	업종을 어느 무리에 넣을지. 축소추정의 '비슷한 무리' 기준이기도 하다	04~08 → PPT
`SCOPE_BY_GROUP`	대형·기관 = large, 제외 = exclude, 의료 = essential	분석 범위. core(소상공인)만 피해 합계에 들어간다	04~08 → PPT

다른 파일에 있는 설정

파일	설정	현재 값	뜻
`04_model_sensitivity.py`	`HAZARDS`	호우·보통비·약한비·강설·적설지속·폭염	모델에 넣는 날씨 변수
`04_model_sensitivity.py`	`LAGS`	호우 다음날·이틀 뒤	순손실에 더하는 '이후' 기간
`04_model_sensitivity.py`	`CAL_FLAGS`	공휴일·추석·연휴 전날·특수일·급여일	날씨가 아닌 매출 변화 원인(통제 변수)
`04_model_sensitivity.py`	`HAC_LAGS`	7	오차 계산 때 며칠간의 연속성을 고려할지
`06_priority_algorithm.py`	`WEIGHTS`	규모 0.4, 손실률 0.4, 회복불가 0.2	우선순위 점수의 비중
`06_priority_algorithm.py`	`MIN_LOSS`	0.03	예상 손실 3% 미만은 지원 대상에서 제외

파일	설정	현재 값	뜻
`06_priority_algorithm.py`	`NON_SUPPORT_GROUPS`	교통·연료, 내구재	손실이 커도 지원하지 않는 업종군
`06_priority_algorithm.py`	`SUPPORT`	업종군별 지원 수단 문구	우선지원 표의 '지원 수단' 칸
`08_sk_flow.py`	`BLOCK_M` / `BUSY_QUANTILE`	250m / 0.70	격자를 묶는 크기 / 유동 상위 30%만 사용
`presentation/build_deck.py`	`TEAM` / `PERSON`	j2tori / 전건우	표지와 파일명

정제는 이 설정들에 따라 아래 순서로 자동 적용된다.

1. 카드 데이터를 cp949로 읽고, 연령 표기 공백을 지운다('20 대' → '20대').
2. 날짜·지역·업종별로 매출을 더하고, 현지·외지·고령(60대 이상) 매출을 따로 나눠 적는다.
3. 184일 전체 달력을 만들어 빈 날을 observed=0, 매출=NaN으로 표시한다.
4. 데이터 1의 법인 카드 매출은 `sales_corp`로 따로 떼어 개인 매출과 섞지 않는다.
5. 기상·달력 정보를 날짜로 이어 붙인다. 연결이 안 된 줄이 있거나 중복이 있으면 03번이 오류를 내며 멈춘다.

4. 회귀분석을 숫자로 이해하기

회귀분석은 "매출을 바꾸는 여러 원인을 동시에 넣고, 각 원인이 따로 얼마나 영향을 주는지" 나눠 주는 도구다. 이 분석은 '지역 × 업종' 129개(춘천 61 + 강남 68, 의료 포함)마다 아래 식을 하나씩 따로 풀었다. 코드는 `04_model_sensitivity.py`의 `fit_series` 함수다.

```
\log(\text{매출}_t) = a + \beta \cdot \text{호우}_t + \gamma_1 \cdot \text{호우}_{t-1} + \gamma_2 \cdot \text{호우}_{t-2} + (\text{다른 날씨}) + (\text{요일·월·공휴일 등}) + \varepsilon_t
```

식의 각 부분은 이렇게 읽는다.

- **log(매출)**: 매출에 로그를 씌우면 효과가 '원'이 아니라 '%'로 나온다. 그래서 매출 규모가 100배 다른 강남 백화점과 춘천 화원을 같은 자로 비교할 수 있다.
- **호우_t**: 그날이 호우일이면 1, 아니면 0. β가 '호우 당일 효과'다.
- **호우_{t-1}, 호우_{t-2}**: 어제·그제가 호우일이었는지. γ가 '비 그친 뒤 효과'다.
- **다른 날씨**: 보통비(10~30mm), 약한비(10mm 미만), 강설, 적설 지속, 폭염
- **요일·월·공휴일 등**: 요일(7개), 월(6개), 공휴일, 추석 연휴, 연휴 전날, 특수일(12/24·12/31), 급여일(10·25일)

- **ε(오차)**: 위 원인으로 설명되지 않는 나머지 매출 변동

실제 예시: 춘천 골프장(실내/실외골프장)

`outputs/sensitivity_series.csv`의 춘천 골프장 줄을 따라 계산해 보면 이렇다.

단계	계산	결과
1. 회귀 계수 β (`b_heavy_rain`)	코드가 추정	-0.814
2. %로 바꾸기 (`gross`)	e^(-0.814) - 1 = 0.443 - 1	**-55.7%**
3. 불확실성 (`se_heavy_rain`)	코드가 추정	0.118
4. 95% 범위	β ± 1.96 × 0.118 = [-1.045, -0.583] → %로 바꾸면	[-64.8%, -44.2%]

즉 "춘천 골프장은 호우일에 매출이 약 56% 줄고, 진짜 값은 44~65% 감소 사이일 가능성이 크다"는 뜻이다. 범위가 0을 포함하지 않으므로 우연으로 보기 어렵다.

왜 e^β - 1인가? 로그 매출이 β만큼 바뀌면 실제 매출은 e^β배가 된다. e^(-0.814) = 0.443배, 곧 평소의 44.3%만 팔렸다는 뜻이고, 줄어든 비율은 1 - 0.443 = 55.7%다. β가 -0.1처럼 작으면 e^β - 1 ≈ β라서 계수를 그대로 %로 읽어도 비슷하다.

표준오차를 'HAC'로 계산한 이유

매출은 어제 잘 팔리면 오늘도 잘 팔리는 경향이 있다. 이런 연속성을 무시하면 불확실성(표준오차)을 실제보다 작게 계산해, 효과가 실제보다 확실해 보인다. HAC는 일주일(7일) 정도의 연속성을 감안해 표준오차를 보수적으로 키우는 방식이다(`HAC_LAGS = 7`).

모델이 한 업종에서 쓰는 날수

매출이 있는 날만 쓴다(observed=1). 대부분 184일을 모두 쓰지만, 춘천 예식장처럼 작은 업종은 마스킹으로 빠진 날이 있어 178일만 쓴다(`n_days` 칸).

5. 축소추정 계산 따라하기

축소추정은 "업종 혼자서 나온 숫자"와 "같은 무리 업종들의 평균"을 섞는데, 업종의 숫자가 불확실할수록 평균 쪽을 더 믿는 방법이다. 코드는 `04_model_sensitivity.py`의 `eb_shrink` 함수다.

$$\text{축소추정값} = w \times \text{업종 추정값} + (1 - w) \times \text{무리 평균}, \quad w = \frac{\tau^2}{\tau^2 + se^2}$$

- **se**: 업종 추정값의 불확실성(표준오차). 매출이 작고 들쭉날쭉한 업종일수록 크다.
- **τ²(타우 제곱)**: 같은 무리 안에서 업종들이 '진짜로' 다른 정도. 업종들의 추정값이 오차만으로 설명되는 것보다 더 퍼져 있으면, 그 초과분을 진짜 차이로 본다(DerSimonian-Laird 방식).
- **w(신뢰 가중치)**: 0~1. 1에 가까우면 업종 숫자를 거의 그대로, 0에 가까우면 무리 평균을 거의 그대로 쓴다.
- **무리**: '지역 × 업종군'. 예: '춘천 × 생활서비스'. 무리에 업종이 3개 미만이면 그 지역 전체 평균을 쓴다.

예시 1: 춘천 예식장 (많이 당겨진 경우)

단계	값	어디서
업종 추정값(호우 당일)	+46.0%	`gross`
불확실성 se	0.450	`gross_se`
무리 평균(춘천 생활서비스 9개 업종)	-5.9%	코드 내부 계산
τ²	0.0192	코드 내부 계산
w	0.0192 ÷ (0.0192 + 0.450²) = **0.087**	`gross_shrink_w`
축소추정값	0.087 × 46.0% + 0.913 × (-5.9%) = 4.0% - 5.4% = **-1.4%**	`gross_eb`

"비 오면 예식장 매출이 46% 늘어난다"는 말이 안 되는 값은 불확실성이 너무 커서(se 0.45) 거의 믿지 않고(w 8.7%), 비슷한 업종들의 평균 쪽으로 당겨졌다. 예식은 미리 잡힌 일정이라 비와 상관없이 며칠의 우연으로 숫자가 튀기 쉽다.

예시 2: 춘천 골프장 (거의 그대로인 경우)

단계	값
업종 추정값	-55.7%
불확실성 se	0.052
w	**0.95**
축소추정값	**-53.6%**

골프장은 매출이 크고 184일 모두 관측돼 불확실성이 작다. 그래서 업종 숫자를 95% 믿고, 거의 움직이지 않았다. 즉 축소추정은 큰 업종의 결과는 건드리지 않고, 작은 업종의 엉뚱한 값만 고친다.

어디에 적용했나

호우 당일 손실률(`gross`), 순손실률(`net`), 적설 지속일(`rate_snow_cover`), 폭염일(`rate_heat33`), 보통비(`rate_rain_mid`) 다섯 가지에 각각 따로 적용했다. 결과는 이름 뒤에 `_eb` 가 붙은 칸에 있다. 춘천 56개 업종의 호우 당일 손실률 편차(표준편차)는 17.7%p에서 10.9%p로 줄었다.

6. 순손실·피해액 계산 따라하기

순손실은 호우 당일과 이후 이틀의 매출 변화율을 더한 값이다. '평상시 하루 매출의 몇 %를 3일에 걸쳐 잃었나'로 읽는다.

```
\text{순손실률} = (e^{\beta} - 1) + (e^{\gamma_1} - 1) + (e^{\gamma_2} - 1)
```

세 업종으로 비교하기

	춘천 한식	강남 할인점·슈퍼마켓	춘천 골프장
호우 당일	-7.2%	-19.8%	-55.7%
다음날	-3.3%	+7.2%	+0.8%
이틀 뒤	-4.5%	+6.1%	-9.1%
순손실(합계)	**-15.0%**	**-6.5%**	**-64.0%**
순손실(축소추정 후)	-13.4%	-7.4%	-53.6%
회복률	-86%	+56%	0%
손실 유형	지속감소형	이연형	소멸형

- **춘천 한식**: 비 온 다음날과 그다음날에도 매출이 더 줄었다. 그래서 3일 합계 손실이 당일보다 두 배 크다.
- **강남 할인점**: 당일에는 20% 줄었지만 다음 이틀에 13% 더 팔렸다. 장보기를 미루던 손님이 돌아온 것이다.
- **춘천 골프장**: 당일에 대부분의 매출이 사라지고 돌아오지 않았다. 취소된 라운드는 다시 잡히지 않는다.

손실 유형과 등급을 정하는 규칙

회복률 = 1 - (순손실 ÷ 당일 손실). 강남 할인점은 1 - (-7.4% ÷ -17.1%) = 56%, 즉 당일 잃은 매출의 절반 이상을 되찾았다. 음수면 회복은커녕 더 잃었다는 뜻이다.

손실 유형	조건 (위에서부터 순서대로 판정)
영향미미	당일 손실(축소추정)이 3% 미만
이연형	회복률 50% 이상
지속감소형	회복률 -20% 미만 (이후에 더 줄었음)
소멸형	회복률 20% 미만
부분회복형	그 외

취약 등급	조건
상	순손실 10% 이상 **그리고** 통계적으로 확실(순손실 ÷ 표준오차 ≤ -1.645)
중	순손실 5% 이상 **또는** 통계적으로 확실
하	그 외

피해액(원) 계산

```
\text{6개월 피해액} = \text{순손실률(축소추정)} \times \text{평상시 하루 매출} \times \text{호우일 수}
```

'평상시 하루 매출'(`baseline_daily`)은 호우일, 그 뒤 이틀, 눈 온 날을 뺀 나머지 날의 평균이다.

업종	순손실률	평상시 하루 매출	호우일	6개월 피해액
춘천 골프장	53.6%	2.71억 원	18일	0.536 × 2.71억 × 18 = **26.2억 원**
춘천 한식	13.4%	12.49억 원	18일	0.134 × 12.49억 × 18 = **30.2억 원**

이렇게 구한 업종별 피해액을 의료를 빼고 모두 더하면 **춘천 187억 원, 강남 273억 원**이다 (`damage_net_won` 칸의 합). 순손실률이 0보다 크면(매출이 늘었으면) 피해는 0으로 본다.

검산은 다른 길로 한다. 춘천 전체 매출로 회귀한 순손실률 14.6% × 전체 평상시 하루 매출 72.8억 × 18일 = 약 192억 원으로, 업종을 하나씩 더한 187억과 거의 같다. 강남은 같은 검산의 95% 범위가 0을 포함해 불확실하다.

7. 검증과 강건성 점검 상세

검증은 두 가지 질문에 답한다. "모델이 처음 보는 비에서도 맞는가"(교차검증)와 "조건을 바꿔도 결론이 유지되는가"(강건성 점검)다.

모의고사: 호우 이벤트 단위 교차검증 (LOEO)

코드는 `04_model_sensitivity.py` 의 `cross_validate` 다.

1. 연속된 호우일을 하나의 '이벤트'로 묶는다. 춘천 14개, 강남 13개다.
2. 이벤트 하나와 그 뒤 이틀을 시험지로 숨긴다.
3. 나머지 날만으로 세 가지 모델을 학습시킨다.
   - A. 달력만: 날씨를 모르는 모델(비교 기준)
   - B. 날씨 포함: 업종별로 따로 학습
   - C. 날씨 포함 + 축소추정: 최종 모델
4. 숨긴 날의 매출을 예측하고 실제와 비교한다. 이를 이벤트 27개 모두에 대해 반복한다.

채점은 **WAPE**로 한다. WAPE = Σ|실제 - 예측| ÷ Σ실제. 예를 들어 실제 100을 80으로, 실제 50을 60으로 예측했다면 (20 + 10) ÷ 150 = 20%다. 매출이 큰 업종의 오차가 더 무겁게 반영된다.

구간	지역	A. 달력만	B. 날씨 포함	C. 축소추정
호우 당일	춘천	19.1%	16.7%	**16.1%**
호우 당일	강남	27.2%	25.3%	**24.3%**
호우 후 1~2일	춘천	14.1%	14.4%	13.6%
호우 후 1~2일	강남	21.3%	22.3%	21.4%

호우 당일에는 C가 확실히 낫다. 호우 후 1~2일은 세 모델이 비슷하고, 이벤트 27개를 하나씩 보면 가장 잘 맞힌 모델이 C 10번, A 9번, B 8번으로 섞여 있다. 이 한계는 발표자료 14번 슬라이드에 적어 두었다. 결과 파일은 `outputs/cv_summary.csv` , 예측값 전체는 `cv_predictions.csv` 다.

강건성 점검 (`05_robustness_checks.py`)

점검	방법	결과
지역 합계로 다시 계산	업종별 숫자를 더하지 않고, 184일 모두 관측된 업종의 지역 매출 합계로 같은 회귀를 한다	춘천 -14.6% [-21.4, -7.9], 강남 -6.7% [-16.9, +3.6]

점검	방법	결과
두 지역 차이 검정	두 지역을 한 모델에 넣고, 같은 주에 두 지역에 함께 오는 충격을 고려한 '주 단위 군집 표준오차'를 쓴다	차이 -8.0%p [-14.8, -1.1], p=0.022
추석 기간 제외	10/3~10/12를 빼고 위 두 검사를 다시 한다	춘천 -13.1%, 강남 -4.0%, 차이 -9.1%p, p=0.045
마스킹: 하루 통째 빠진 날	호우일에 '업종-날짜'가 통째 빠질 확률이 늘어나는지	춘천 +0.13%p(p=0.77), 강남 +0.19%p(p=0.55) → 문제없음
마스킹: 지워진 작은 칸	지워진 칸이 1~4건이었다고 가정해 '숨은 매출'을 호우일과 평상일에 비교	손실이 부풀려졌을 수 있는 폭: 춘천 0.5~1.5%p, 강남 0.2~0.6%p

대괄호 안은 95% 범위다. 춘천 범위는 0을 포함하지 않아 확실하고, 강남은 0을 포함해 불확실하다. 두 지역의 차이는 0을 포함하지 않아 "춘천이 더 크게 당한다"는 결론은 유지된다. 결과 파일은 `outputs/robust_region_net.csv`, `robust_group_net.csv`, `robust_masking.csv`다.

8. 우선지원 알고리즘 점수 계산 따라하기

알고리즘은 예보 하나를 받아 업종마다 점수를 매기고, 그 점수와 확실성으로 등급을 정한다. 코드는 `06_priority_algorithm.py`의 `rank_scenario`다.

1단계: 예보로 예상 손실률 고르기

예보 조건	쓰는 값
강수 30mm 이상	순손실률(축소추정, `net_eb`)
강수 10~30mm	보통비 손실률(`rate_rain_mid_eb`)
적설 0.2cm 이상	적설 지속일 손실률을 더함(`rate_snow_cover_eb`)
최고기온 33℃ 이상	폭염일 손실률을 더함(`rate_heat33_eb`)

예상 손실액(하루) = 예상 손실률 × 평상시 하루 매출이다.

2단계: 세 가지 점수를 0~1로 만들기

'춘천 호우 60mm' 시나리오의 1위, 놀이시설(종합레저타운)의 계산이다.

점수	계산	놀이시설
피해 규모	log(1 + 예상 손실액) ÷ 지역 내 최댓값. 최댓값은 한식(하루 1.68억)	log(3,521만) ÷ log(1.68억) = 17.38 ÷ 18.94 = **0.918**
피해 강도	예상 손실률 ÷ 지역 내 최댓값	65.2% ÷ 65.2% = **1.000** (놀이시설이 최대)
회복 불가도	(1 - 회복률), 최대 1.5로 자른 뒤 ÷ 1.5	1 - (-0.54) = 1.54 → 1.5 → **1.000**
최종 점수	0.4 × 규모 + 0.4 × 강도 + 0.2 × 회복불가	0.367 + 0.400 + 0.200 = **0.967**

피해 규모에 log를 씌운 이유는 업종 간 금액이 수백 배 차이 나기 때문이다. 그대로 쓰면 매출이 큰 한식·편의점만 상위에 오고, 비율로 크게 당하는 작은 업종이 묻힌다. 실제로 춘천 한식은 손실액이 가장 크지만 손실률이 13%라 6위다.

3단계: 등급 정하기

1. **확실한가**: 예상 손실률 ÷ 표준오차 ≤ -1.645(한쪽 95% 기준)이고 손실률이 3% 이상인가
2. **지원 대상인가**: 교통·연료, 내구재가 아니고, 손실 유형이 '이연형'이 아닌가. 의료는 처음부터 빠져 있다
3. 두 가지를 모두 만족하는 업종 중 점수 상위 25% = **1순위**, 나머지 = **2순위**, 만족하지 못하면 **모니터링**

그래서 점수가 높아도 모니터링이 될 수 있다. 강남 자동차서비스는 점수 0.77로 3등이지만 손실이 통계적으로 확실하지 않아 모니터링이다.

검증 두 가지

- **백테스트**: 7장의 모의고사 예측에서, 숨긴 호우마다 '예측 손실률 상위 10개 업종'과 '실제 손실률 상위 10개'가 몇 개 겹치는지 세었다(precision@10). 춘천은 평균 33%, 무작위로 고르면 10 ÷ 56 = 18%다. 강남은 19% vs 16%로 차이가 작다. 결과: `outputs/priority_backtest.csv`
- **가중치 민감도**: 0.4/0.4/0.2 대신 무작위 가중치 500개를 넣어 봤다. 상위 10개 업종이 평균 춘천 82%, 강남 92% 유지됐다. 가중치를 어떻게 정하든 결과가 크게 바뀌지 않는다는 뜻이다. 결과: `outputs/priority_weight_sensitivity.csv`

9. SK 방문형 상권 지수 계산

방문형 지수는 "주말에 사람이 평일보다 얼마나 많은가"다. 1보다 크면 주말에 찾아오는 상권, 작으면 평일 직장인이 많은 상권이다. 코드는 `08_sk_flow.py`다.

1. **중복 제거**: 12월 파일의 중복 12.1만 줄을 지운다.
2. **평일·주말 평균**: 칸마다 월~금 5개 값의 평균(평일)과 토·일 2개 값의 평균(주말)을 구한다.

3. **250m 블록으로 묶기**: 50m 칸 여러 개를 250m 정사각형 하나로 묶어 더한다. 50m 칸은 값이 작고 들쭉날쭉해서다.
4. **6개월 모두 있는 블록만** 남긴다.
5. **사람이 많은 상위 30% 블록만** 남긴다. 한적한 곳은 주말 값이 0으로 비어 있는 경우가 많아(춘천 칸의 6~8%) 지수가 왜곡된다.
6. **지수 = 주말 합 ÷ 평일 합**

예를 들어 강남에서 유동인구가 가장 많은 블록은 평일 합 986,243, 주말 합 570,940이다. 지수 = 570,940 ÷ 986,243 = **0.58**, 주말에 사람이 평일의 58%뿐인 전형적인 직장형 상권이다.

	춘천	강남
분석 블록 수	3,223	210
지수 중간값	1.11	0.69
방문형(지수 > 1) 비율	57%	13%

춘천 블록 수가 훨씬 많은 것은 춘천시 면적이 강남구의 약 28배여서다.

이 지수를 '방문형'으로 봐도 되는 근거

카드 데이터에서 외지 고객 매출 비중을 평일과 휴일(주말·공휴일)에 비교했다. 춘천은 16.4% → 22.9%로 휴일에 크게 오르고, 강남은 40.4% → 45.2%로 거의 그대로다. 주말에 사람이 많은 곳 = 외지에서 찾아오는 곳이라는 해석을 뒷받침한다.

확인하지 못한 것

"방문형 블록일수록 비가 많이 온 달에 사람이 더 줄었나"도 검사했지만 효과가 없었다(춘천 p=0.64). 유동인구가 월 평균이라 날짜별 비와 연결할 수 없기 때문이다. 그래서 SK 데이터는 '어디가 방문형인가'를 보여주는 데만 썼다. 결과 파일은 `outputs/sk_block_exposure.csv`, `sk_did.csv`다.

9-1. 지하철 도착 인원과 원인 분해

코드는 `10_subway_flow.py`다. 카드 매출만으로는 '안 와서 줄었는지, 와서 덜 써서 줄었는지' 구분할 수 없어서, 날짜별 도착 인원을 따로 봤다.

1단계: 호우가 도착 인원을 줄이는가 (`arrival_effects`)

4장의 매출 회귀와 같은 모양이다. 왼쪽이 매출 대신 log(하차 인원)이고, 오른쪽은 호우 당일·1일 뒤·2일 뒤 + 요일 + 월 + 공휴일 + 추석 표시다. 계수 b는 e^b - 1로 % 변화가 되고, 표준오차는 HAC(7일)다. 같은 식을 평일·휴일로 나눠서도 돌린다.

호우 당일 도착 인원	춘천	강남
전체 기간	-14.4%	-6.7% [-16.2, +3.9]
추석 제외 (발표 기준)	-13.7% [-15.9, -11.5]	-0.4% [-2.3, +1.5]
추석 제외, 평일 / 휴일	-13.7% / -18.1%	-0.6% / -3.2%

강남 전체 기간 -6.7%는 추석 연휴 중 호우 하루가 끌어내린 값이다. 평일·휴일로 나눠 보면 어느 쪽도 -3% 안쪽이라, 발표는 두 지역 모두 추석 제외 값을 썼다.

2단계: 매출 감소 중 '발길' 몫 (`mediation`)

매출은 05장과 같은 기준(184일 완전관측 소상공인 업종 합계)이다. 같은 회귀를 두 번 돌린다.

```
\text{발길 몫} = 1 - \frac{b_1\ (\text{도착 인원을 넣은 뒤 호우 계수})}{b_0\
(\text{넣기 전 호우 계수})}
```

춘천(추석 제외)은 호우 매출 효과가 -9.1%에서 도착 인원을 넣으면 -4.1%로 줄어, 발길 몫이 56%다. 강남은 -3.4% → -3.2%로 거의 안 변해 6%다. 범위는 주 단위로 날짜를 다시 뽑는 부트스트랩 500회 (`N_BOOT`)의 2.5~97.5% 구간이며, 춘천은 [28%, 90%]다. 외지 고객 매출(-20.9%)은 45%가 발길 몫이다.

이 계산은 도착 인원과 매출에 함께 영향을 주는 다른 요인을 걸러내지 못해, 엄밀한 인과 분해가 아닌 근사치다. 심사에서는 '발길 감소로 설명되는 몫'이라고 표현한다.

결과 파일은 `outputs/subway_daily.csv` (지역·날짜별 도착·매출), `subway_effects.csv` , `subway_mediation.csv` 이고 그래프는 F12(도착 vs 매출), F13(발길 몫)이다.

9-2. 호우 지수형 손실보전

코드는 `11_index_payout.py` 다. 새 모델을 만들지 않고, 06장의 우선지원 계산을 '비 30mm' 조건으로 돌려 지급표를 만든다.

1. **대상 고르기**: `rank_scenario` 를 강수 30mm로 실행해 등급이 '모니터링'이 아닌 업종(손실 확실 + 지원 대상 업종군)만 남기고, 손실 유형이 이연형·영향미미인 업종을 뺀다(`EXCLUDE_TYPES`).
2. **지급률**: 예상 3일 순손실률(`net_eb`) × 보전율 50%(`COVER`).
3. **지급액**: 지급률 × 평상시 하루 매출(`baseline_daily`). 한 번 발동할 때의 업종 합계가 '1회 지급액'이고, 여기에 호우일 수를 곱하면 6개월 예산이다.

예를 들어 춘천 골프장은 지급률 26.8%, 한 번에 업종 전체 약 7,276만 원이고, 춘천 한식은 지급률 6.7%지만 매출이 커서 약 8,398만 원이다. 업종별 값은 `outputs/payout_table.csv` 에 있다.

	춘천	강남
지급 대상 / 분석 업종	30 / 56	8 / 63
1회 지급액	4.15억	1.35억
호우일 수	18일	15일
6개월 예산	74.6억 (순손실 187억의 39.9%)	20.2억 (7.4%)
지급률 중앙값	9.8%	7.2%

기초위험 검증 (`backtest`)

지수형은 피해를 조사하지 않으므로, 지급액과 실제 손실이 얼마나 어긋나는지가 핵심 약점이다. 7장 모의고사의 예측(`cv_predictions.csv`)을 그대로 쓴다. 지급 대상 업종만 모아, 호우 이벤트마다 다음 둘을 비교한다.

- 지급액 = (달력 모델 예측 - 날씨 모델 예측) × 50% — 학습에서 뺀 이벤트에 대해 미리 정해지는 돈
- 실손 = (달력 모델 예측 - 실제 매출) × 50%

춘천 14건은 상관 0.74, 총지급/총실손 83%이고, 강남 13건은 0.57, 39%다. 실손이 0 아래인(비가 와도 매출이 줄지 않은) 이벤트가 두 지역 각각 4건 있다. 결과는 `payout_backtest.csv` , 그래프는 F14다.

풍수해보험이 재산(건물·시설·재고)만 보상한다는 설명은 웹 검색으로 확인한 제도 배경이며, 분석에는 쓰이지 않는다.

9-3. 기준 민감도와 머신러닝 비교 (점검용)

세 스크립트는 새 결론을 만들지 않고, 기존 결론이 설정값에 기대지 않는지 확인한다. 결과는 발표 슬라이드 17·22와 예상 질문 Q46~Q51에 쓰였다.

호우 기준 민감도 (`12_threshold_sensitivity.py`)

호우 표시만 바꾸고(30·40·50mm, 호우주의보급 = 3시간 60mm 또는 12시간 110mm), 보통비 구간은 '10mm 이상이면서 호우가 아닌 날'로 다시 만든 뒤 05장의 지역 합계 회귀와 10단계의 도착 인원 회귀를 다시 돌린다. 주의보급은 시간 자료의 3·12시간 누적 최대로 판단한다.

기준	호우일(춘천/강남)	춘천 순손실	강남 순손실	차이 p값	도착(춘천/강남)
30mm	18/15	-14.6%	-6.7%	0.022	-13.7% / -0.5%
40mm	12/10	-12.5%	-5.0%	0.29	-13.5% / -1.0%
50mm	7/6	-16.1%	-12.3%	0.35	-15.6% / -0.7%
호우주의보급	1/1	추정 불가	추정 불가	0.43	—

30mm 줄이 기존 발표 숫자와 정확히 같아 코드가 맞다는 것도 확인된다. 결과는 `outputs/threshold_sensitivity.csv`.

분석 대상 업종 기준 민감도 (`13_count_threshold_sensitivity.py`)

관측일 90% 조건은 두고 하루 결제 중간값 기준만 필터 없음·5·10·20·30·50·100건으로 바꾼다. 04의 업종별 회귀·축소추정과 05의 지역 회귀를 다시 돌린다. 어떤 기준에서도 춘천 순손실 -14.6%, 지역 차이 -8.0%p(p=0.02)이고 피해액은 184~188억 / 270~275억이다. 필터를 없애면 '상' 등급 업종이 32개에서 36개로 늘어 잡음이 커진다. 결과는 `outputs/count_threshold_sensitivity.csv`.

머신러닝 비교 (`14_ml_benchmark.py`)

04의 교차검증과 같은 폴드·같은 행(5,581개)에서 그래디언트 부스팅(scikit-learn `HistGradientBoostingRegressor`)을 돌린다.

- 회귀는 업종마다 따로 학습하지만, 부스팅은 한 지역의 모든 업종을 한 번에 학습한다.
- 목표값은 log(매출) − 업종 평균 log(매출), 입력은 강수량·전날 강수량·최고기온·호우 표시와 시차·달력·업종군·업종이다.
- 하이퍼파라미터는 코드 맨 위 `PARAMS`에 고정되어 있다(튜닝 없음).

호우 당일 WAPE	달력만	+기상(개별)	+기상(축소추정)	부스팅
춘천	19.1%	16.7%	**16.1%**	16.9%
강남	27.2%	25.3%	24.3%	**20.6%**

호우 후 1~2일은 춘천 15.1%(회귀 13.6%), 강남 18.6%(회귀 21.4%)다. 14를 돌린 뒤 07을 다시 실행하면 F6에 부스팅 막대가 들어간다(`cv_ml_summary.csv`가 있을 때만).

10. 발표 숫자의 출처표

발표자료의 핵심 숫자는 모두 `outputs/report_numbers.csv` 한 파일에 모여 있다. `src/09_report_numbers.py`를 실행하면 다시 만들어지고, 슬라이드 번호(`slide` 칸)로 찾을 수 있다. 숨어있는 숫자 없이 모두 코드로 재현된다.

슬라이드	발표 숫자	정확한 값	원래 출처
1·2·7	춘천 3일 순손실 -14.6%	-14.636% [-21.4, -7.9]	05 지역 합계 회귀
2·7	강남 3일 순손실 -6.7%	-6.676% [-16.9, +3.6]	05 지역 합계 회귀
2·7	두 지역 차이 -8.0%p, p=0.022	-7.960%p [-14.8, -1.1]	05 주 단위 군집 검정
7	추석 제외 시 -9.1%p, p=0.045	-9.131%p [-18.1, -0.2]	09에서 05 함수를 다시 실행
6	폭염일 춘천 0.0%, 강남 -0.9%	+0.004%, -0.871%	07 `region_fit`
6	호우일 춘천 -10.7%	-10.681% [-14.5, -6.8]	05 지역 합계 회귀
9	춘천 외지 -22.4%, 현지 -7.7%	-22.376%, -7.685%	07 `region_fit`
9	강남 현지 -9%, 외지 -8%	-9.380%, -8.242%	07 `region_fit`
2·10	호우일 도착 인원 춘천 -13.7%, 강남 -0.4%	-13.701% [-15.9, -11.5], -0.420% [-2.3, +1.5] (추석 제외)	10 `subway_effects.csv`
10	춘천 휴일 도착 -18.1%	-18.110% (추석 제외, 휴일)	10 `subway_effects.csv`
2·11	매출 감소 중 발길 몫 춘천 56%, 강남 6%, 외지 45%	55.868% [28.0, 90.1], 5.5%, 44.610%	10 `subway_mediation.csv`
12	방문형 블록 춘천 57%, 강남 13%	57.0%, 12.9%	08 `sk_block_exposure.csv`
12	휴일 외지 비중 춘천 16.4% → 22.9%	16.425% → 22.893%	03 `panel_daily.csv`
14	춘천 여가·숙박 -39%	업종군 순손실	05 `robust_group_net.csv`
15	6개월 순손실 춘천 187억, 강남 273억	187.2억, 273.1억	04 `damage_net_won` 합
15	매출 대비 1.42% vs 0.43%, 3.3배	1.416%, 0.432%	09

슬라이드	발표 숫자	정확한 값	원래 출처
15	검산 192억 [103~281]	191.8억 [102.9, 280.6]	09
16	업종 간 편차 17.7 → 10.9%p	17.656 → 10.859	04 `gross`, `gross_eb`
17	호우 당일 WAPE 춘천 19.1 → 16.1%	19.110 → 16.100	04 `cv_summary.csv`
17	호우 당일 WAPE 강남 27.2 → 24.3%	27.198 → 24.266	04 `cv_summary.csv`
17	부스팅 WAPE 춘천 16.9%, 강남 20.6%	16.864, 20.572	14 `cv_ml_summary.csv`
22	강남 백테스트 19% vs 무작위 16%	19.231% vs 15.934%	06 `priority_backtest.csv`
18	백테스트 33% vs 무작위 18%	32.857% vs 17.904%	06 `priority_backtest.csv`
18	가중치 변경 시 82~92% 유지	81.660%, 92.260%	06 `priority_weight_sensitivity.csv`
2·19	지급 대상 30개, 1회 4.1억, 6개월 75억(순손실의 40%)	30, 4.145억, 74.603억, 39.863%	11 `payout_budget.csv`
2·20	지급액-실손 상관 춘천 0.74(83%), 강남 0.57(39%)	0.743 (83.0%), 0.573 (39.2%)	11 `payout_backtest.csv`
21	지급률 여가·숙박 24%, 외식·생활소매 7~10%	23.869%, 6.791%, 10.000% (춘천 중앙값)	11 `payout_table.csv`
3·4	업종 90개 → 64개	90, 64	02 `industry_coverage.csv`
4	마스킹 과대추정 최대 1.5%p	숨은 매출 비중의 호우일-평상일 차이	05 `robust_masking.csv`

참고로 이 출처표를 만드는 과정에서, 초기에 임시로 계산했던 '강설일 춘천 외지 고객 -10.9%'가 최종 모델로는 -4.95%(p=0.37, 유의하지 않음)로 나오는 것을 발견했다. 그래서 9번 슬라이드에서 빼고 강남 비교로 바꿨다.

11. 수정 가이드

무엇을 바꾸든 순서는 같다. **설정을 고친다 → 영향받는 코드를 순서대로 다시 실행한다 → `report_numbers.csv` 에서 새 숫자를 확인한다 → 발표자료의 문구를 고친다.** 발표자료의 숫자는 `build_deck.py` 에 글자로 직접 적혀 있어 자동으로 바뀌지 않는다는 점이 가장 중요하다. 그래프는 자동으로 바뀐다.

바꾸고 싶은 것	고칠 곳	다시 실행
호우 기준(30mm → 다른 값)	`config.py` `HEAVY_RAIN_MM`	01 → 03 → 04 → 05 → 06 → 07 → 08 → 10 → 11 → 09 → PPT
폭염 기준(33℃)	`config.py` `HEAT_TMAX`	01 → 03 → 04~09 → PPT
공휴일 추가·수정	`config.py` `HOLIDAYS`	03 → 04~09 → PPT
업종을 다른 업종군으로 옮기기	`config.py` `INDUSTRY_GROUPS`	04~09 → PPT
의료를 소상공인 합계에 넣기	`config.py` `SCOPE_BY_GROUP` 에서 "의료" 지우기	04~09 → PPT (강남 피해액이 크게 늘어난다)
분석 대상 기준(관측률·건수)	`config.py` `MIN_COVERAGE` , `MIN_MEDIAN_CNT`	02 → 03 → 04~09 → PPT
순손실 기간(2일 → 3일)	`04_model_sensitivity.py` 의 `LAGS` 에 `heavy_rain_lag3` 추가 + `05_robustness_checks.py` 의 `NET_LAGS`	04~09 → PPT (그래프 축 이름도 수정)
우선순위 가중치	`06_priority_algorithm.py` `WEIGHTS`	06 → 07 → 09 → PPT
지원 수단 문구	`06_priority_algorithm.py` `SUPPORT`	06 → PPT
예보 시나리오 추가(예: 강남 80mm)	`06_priority_algorithm.py` `SCENARIOS`	06 (그래프가 필요하면 07의 `fig_priority` 에 시나리오 이름 지정)
머신러닝 설정값	`14_ml_benchmark.py` `PARAMS` , 입력 변수 `NUM` · `CAT`	14 → 07 → 09 → PPT
민감도 점검 기준 추가	`12_threshold_sensitivity.py` `RULES` , `13_count_threshold_sensitivity.py` `RULES`	12 또는 13 (발표 숫자에는 영향 없음)
보전율(50%)	`11_index_payout.py` `COVER`	11 → 09 → PPT (슬라이드 19·20 숫자)

바꾸고 싶은 것	고칠 곳	다시 실행
지급에서 빼는 손실 유형	`11_index_payout.py` `EXCLUDE_TYPES`	11 → 09 → PPT
지하철 역 목록	`10_subway_flow.py` `CHUNCHEON`, `GANGNAM`	10 → 09 → PPT (슬라이드 10·11·13 숫자)
그래프 색·문구	`07_figures.py`의 각 `fig_` 함수	07 → PPT
슬라이드 문구·숫자·발표자 노트	`presentation/build_deck.py` (슬라이드 순서대로 적혀 있음)	PPT
표지 팀명·성명	`build_deck.py` `TEAM`, `PERSON`	PPT
분석 기간 변경	`config.py` `ANALYSIS_*`, `WEATHER_*` (+ 기상청 인증키)	전체

실행 방법

프로젝트 폴더에서 아래 명령을 필요한 번호부터 차례로 실행한다. Python 경로는 `C:\Users\SAMSUNG\miniconda3\python.exe`다.

```
python src/01_fetch_weather.py      # 기상 (저장된 원본 사용, 수 초)
python src/02_build_card_panel.py   # 카드 정제 (가장 느림, 1~2분)
python src/03_build_panel.py        # 패널 결합
python src/04_model_sensitivity.py  # 모델·교차검증 (1분 내외)
python src/05_robustness_checks.py  # 강건성 점검
python src/06_priority_algorithm.py # 우선지원
python src/14_ml_benchmark.py       # 머신러닝 비교 (07보다 먼저: F6에 들어감)
python src/07_figures.py            # 그래프 F1~F9, F11
python src/08_sk_flow.py            # SK 지수·지도 F10
python src/10_subway_flow.py        # 지하철 도착·발길 분해 F12·F13 (부트스트랩, 1~2분)
python src/11_index_payout.py       # 지수형 보전 지급표·백테스트 F14
python src/09_report_numbers.py     # 발표 숫자 모음 (10·11·14 다음에)
python src/12_threshold_sensitivity.py        # (점검) 호우 기준 민감도
python src/13_count_threshold_sensitivity.py  # (점검) 업종 기준 민감도
python presentation/build_deck.py   # PPT
```

01번은 `KMA_API_KEY` 환경변수가 비어 있으면 멈춘다. 저장된 원본만 다시 쓸 때는 아무 값이나 넣고 실행해도 된다(예: PowerShell에서 `$env:KMA_API_KEY="cached"`). PPT를 만든 뒤 PDF는 PowerPoint에서 '다른 이름으로 저장 → PDF'로 만든다.

문구만 고칠 때

분석은 그대로 두고 슬라이드 문장만 바꾸려면, PowerPoint에서 직접 고쳐도 된다. 다만 그 뒤 `build_deck.py`를 다시 실행하면 직접 고친 내용이 덮어쓰여 사라진다. 두 방식 중 하나만 쓰는 것이 안전하다. 한글은 PowerPoint가 글자 단위로 줄을 바꾸므로, 문장이 길어지면 '64/개'처럼 단어가 잘리지 않는지 확인해야 한다.

12. 주의할 함정

아래는 수정하다가 결론을 잘못 바꾸거나 심사에서 지적받기 쉬운 지점이다.

숫자를 다룰 때

- **빈 날을 0으로 채우지 않는다.** 마스킹으로 빠진 날을 0으로 두면 그날 매출이 100% 줄어든 것으로 계산돼 손실이 크게 부풀려진다.
- **작은 업종의 축소추정 전 숫자를 인용하지 않는다.** 항상 `_eb`가 붙은 칸을 쓴다. 예식장 +46% 같은 값이 그대로 발표에 나가면 안 된다.
- **%와 억 원의 출처를 섞지 않는다.** 발표의 %(-14.6% 등)는 '지역 합계 회귀', 억 원(187억 등)은 '업종별 합산'에서 나온다. 두 방법은 춘천에서 거의 같지만(192억 vs 187억) 강남에서는 차이가 크다.
- **업종 하나의 p값을 과신하지 않는다.** 업종 60~70개를 동시에 검사하면 효과가 없어도 3~4개는 우연히 p<0.05가 나온다. 발표는 지역·업종군 단위 결과를 중심으로 한다.
- **기상 자료가 기상청 것인지 확인한다.** `weather_daily.csv`의 `source` 칸이 'KMA ASOS'여야 한다. 'Open-Meteo'는 개발용이라 극한 강수를 줄여 반영한다.

해석할 때

- **"강남은 회복된다"고 단정하지 않는다.** 강남의 비 그친 뒤 +2.2%는 통계적으로 확실하지 않다. 확실한 것은 '두 지역의 차이'와 '춘천의 손실'이다.
- **폭염 무효과의 범위를 지킨다.** '오프라인 카드 매출'에서 '여름 안의 더 더운 날'을 비교한 결과다. 폭염이 피해가 없다는 뜻은 아니다.
- **눈 관련 결과는 참고용이다.** 강설일은 지역별 2~3일뿐이고, 강남은 12월 학원 일정과 겹친다.
- **특수한 기간과 겹치는지 그래프로 확인한다.** 춘천의 큰 비 3일이 추석 직후여서 별도 점검이 필요했다. 기준을 바꾸면 부록 A 그래프(F8)로 다시 확인한다.
- **지하철 숫자는 추석 제외 값이 발표 기준이다.** subway_effects.csv의 sample 칸이 '전체'인 줄(강남 -6.7%)을 가져다 쓰면 '강남도 발길이 줄었다'는 틀린 말이 된다. 발길 몫 56%는 인과 분해가 아닌 근사치다.
- **지수형 예산 75억은 2025년 하반기 호우 18일 기준이다.** 연간 예산이라고 말하면 안 되고, 도입 시 수년치 강수로 다시 산정해야 한다고 말한다.
- **그래프 제목은 결과보다 강하게 쓰지 않는다.** 마감 전 점검에서 F2('춘천은 이틀간 낮은 상태 유지' → '반등이 없다', 이후 2일 -4.0% [-8.8, +0.9])와 F8('큰 비가 온 날마다' → '18일 중 17일', 7월 20일 예외)을 고쳤다. F8 제목은 개수를 코드가 세어 넣으므로 숫자를 손으로 고치지 않는다. F7의 금액은 하루 매출이 아니라 예상 하루 손실액이다.

- **부스팅이 강남에서 더 정확하다고 지원 기준을 부스팅으로 바꾸지 않는다.** 지원 대상·지급률에는 업종별 손실률과 신뢰구간이 필요하다. 발표의 입장은 '예측은 병용, 기준은 회귀'다.

파일을 다룰 때

- **`build_deck.py`를 다시 실행하면 PowerPoint에서 직접 고친 내용이 사라진다.**
- **그래프 문구에 유니코드 마이너스(−)를 쓰지 않는다.** 맑은 고딕에 글자가 없어 깨진다. 키보드의 하이픈(-)을 쓴다.
- **제공 데이터 원본은 제출 압축 파일에 넣지 않는다.** 대회 목적 외 사용 금지 규정이 있다. 외부 데이터(기상청 원본 `data/external/kma_asos/`)는 넣어야 한다.
- **기상청 인증키를 코드나 문서에 적지 않는다.** 환경변수로만 넣는다.